

14 CHAPTERS

Introduction to DevOps & Cloud

A complete beginner-to-intermediate guide

Aryashree Pritikrishna

github.com/aryashreep/learn-devops-in-90-days

• Linux & Scripting • CI/CD Pipelines • Docker & K8s • Cloud (AWS/Azure/GCP) & Terraform • Monitoring & SRE

TABLE OF CONTENTS

Chapter 01	Software Development & IT Operations
Chapter 02	Problems with Traditional IT (Silos & Slow Releases)
Chapter 03	What is DevOps
Chapter 04	DevOps Culture & Principles
Chapter 05	The DevOps Lifecycle
Chapter 06	CI/CD Concept (High Level)
Chapter 07	Automation & Infrastructure as Code (IaC)
Chapter 08	What is Cloud Computing
Chapter 09	Why Cloud is Needed
Chapter 10	Cloud Service Models (IaaS, PaaS, SaaS)
Chapter 11	Cloud Deployment Models
Chapter 12	DevOps + Cloud Together
Chapter 13	Real-World DevOps & Cloud Workflow
Chapter 14	Career Path Overview

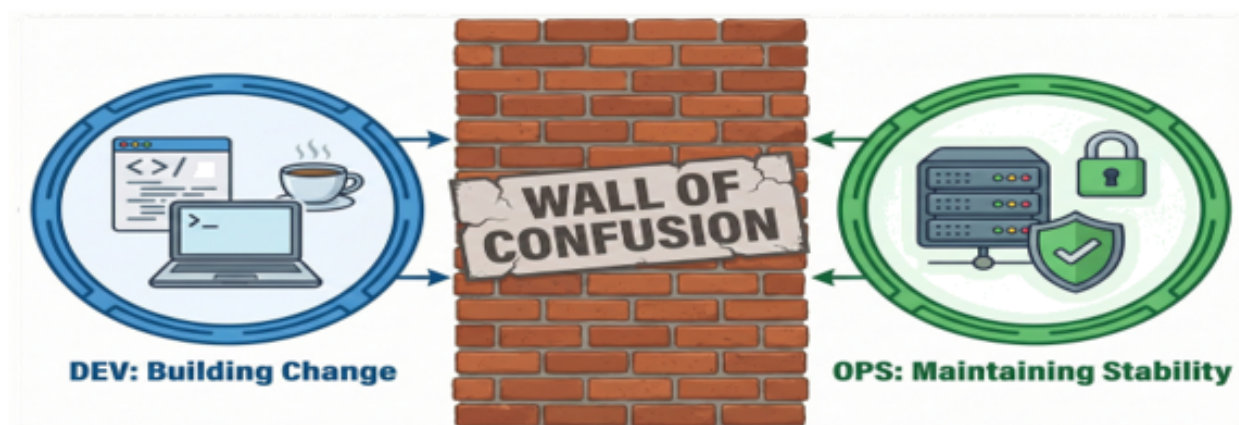
CHAPTER 01

Software Development & IT Operations

01

1. Simple Explanation

Think of a software company as a restaurant. **Software Development (Dev)** are the Chefs — they create new recipes and constantly want to experiment with new ingredients (features). **IT Operations (Ops)** are the Waiters and Hall Managers — they serve the food to customers, ensure the tables are clean, the lights work, and the restaurant does not crash during rush hour.



The Wall of Confusion separating Development and Operations teams

2. Why This Matters in Real IT

- If Devs write code but have nowhere to run it, the software is useless.
- If Ops runs servers but has no software to update, the business stagnates.
- Both teams are essential — but they must work together to deliver value.

3. Key Concepts

Dev (Development): The team responsible for writing code, designing features, and fixing bugs.

Ops (Operations): The team responsible for servers, networks, security, scaling, and keeping the application running 24/7.

SLA (Service Level Agreement): A contract defining uptime targets — Ops must ensure these are met even as Dev pushes changes.

MTTR (Mean Time to Recovery): How fast the team can restore service after an outage. DevOps aims to minimise this.

4. Real-World Example

A bank's mobile app team (Dev) writes a new feature to deposit cheques via camera. The infrastructure team (Ops) manages the servers that process those images and store the financial data securely. Both must coordinate — a release that breaks server capacity is as dangerous as a bug in the code.

■ Common Beginner Confusions

Confusion: Are Ops just customer support?

Correction: No. Ops engineers are highly technical. They manage the machinery (servers, cloud, networking) that runs the software — not people answering phone calls.

✓ Quick Revision Summary

Dev builds the product (focus on change and speed).

Ops runs the product (focus on stability and reliability).

Traditionally they had conflicting goals — DevOps resolves this.

Both are essential for successful software delivery.

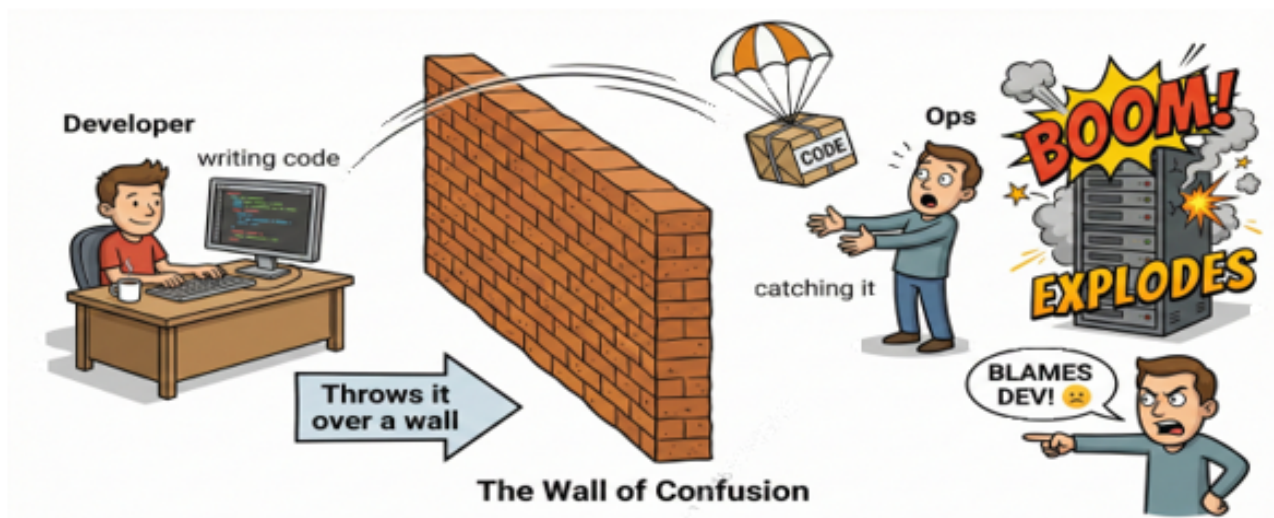
CHAPTER 02

Problems with Traditional IT

02

1. Simple Explanation

In the past, Developers would write code and *throw it over the wall* to Operations. Devs would say 'It works on my machine!' and Ops would say 'It broke the server!' This caused fights, delays, and a blame game whenever the website went down.



Traditional Model: Lack of Communication & High Risk of Failure

2. Why This Matters

- Traditional methods like Waterfall meant software was released very slowly — sometimes once every 6 months or a year.
- In today's world, companies like Netflix or Amazon need to release updates thousands of times a day.
- You cannot do that if teams are fighting.

3. Key Concepts

Silos: Teams working in isolation, not sharing information or tools.

Wall of Confusion: The barrier where Devs do not understand Ops problems, and Ops do not understand the code.

Bottlenecks: Manual processes like manual testing or manual server provisioning that slow everything down.

Waterfall: The sequential development model — Requirements, Design, Code, Test, Deploy. Each step waits for the previous one.

4. Real-World Example

Before DevOps, a large retailer might wait until Black Friday to update their website. Because Dev and Ops had not practised together, the site crashed when millions of users logged in — costing millions in lost sales.

✓ Quick Revision Summary

Traditional IT separates Dev and Ops — creating the Wall of Confusion.

Releases are slow, risky, and painful under Waterfall.

The 'It works on my machine' problem is endemic in siloed organisations.

Manual processes are the enemy of speed and quality.

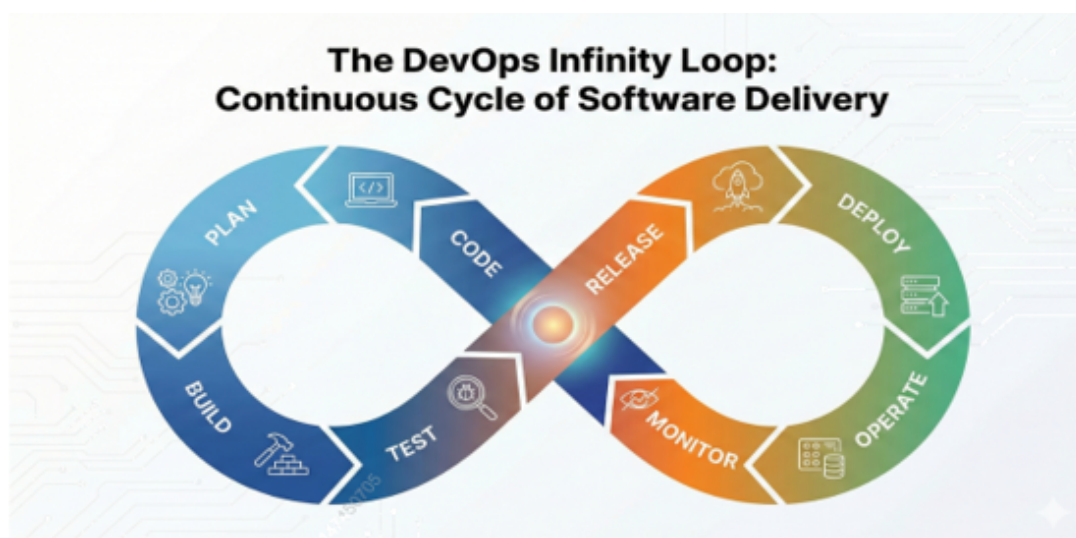
CHAPTER 03

What is DevOps

03

1. Simple Explanation

DevOps is not just a tool — it is a **culture and a methodology**. It combines Development and Operations into one smooth, collaborative team. Instead of 'You build it, I run it,' the mindset shifts to 'You build it, you run it.' Everyone is responsible for the success of the software from idea to production.



The DevOps Infinity Loop: Continuous Cycle of Software Delivery

2. Why This Matters

- It bridges the gap between coding and deployment — no more throwing code over walls.
- It allows companies to release software faster (Speed) and with fewer bugs (Quality).
- It increases customer satisfaction because fixes and features happen quickly.
- Amazon achieved 11.7-second deployment cycles using DevOps — from code commit to production.

3. Key Concepts

Collaboration: Dev and Ops work together daily, not just on release day.

Shared Responsibility: Everyone owns the quality and availability of the product.

Automation: Replacing manual human work with computer scripts and pipelines.

Continuous Improvement: Always measuring, learning, and making things better.

4. Real-World Example

Amazon used to deploy once every few months. After adopting DevOps, they moved to releasing code every 11.7 seconds. Netflix deploys hundreds of times a day using the same principles.

■ Common Beginner Confusions

Confusion: DevOps is a specific job title or a person.

Correction: DevOps is a methodology or culture that the whole organisation must adopt.

One person cannot do DevOps alone.

✓ Quick Revision Summary

DevOps = Development + Operations working as one.

It is a culture and methodology, not just a tool.

Goal: Deliver value to customers faster and more reliably.

Breaks down silos and enables continuous delivery.

CHAPTER 04

DevOps Culture & Principles

04

1. Simple Explanation

DevOps culture is built on trust, communication, and a willingness to learn. It encourages trying new things and not being afraid to fail — as long as you learn from it. Automation is treated as a best friend to remove boring, repetitive work.



People First: Culture and Collaboration, Supported by Automation and Tools

2. The C.A.L.M.S. Framework

Principle	What it means in practice
C — Culture	Communication, trust, and psychological safety. People over process.
A — Automation	Automate repetitive manual tasks — testing, deployment, provisioning.
L — Lean	Minimise waste and wait times. Small batches, fast feedback.
M — Measurement	Use data to improve. Track deployment frequency, MTTR, error rates.
S — Sharing	Share knowledge, tools, and success across teams openly.

3. Real-World Example

Etsy uses **Blameless Post-Mortems**. If an engineer accidentally deletes production data, they do not fire them. They gather the entire team and ask: How can we change the system so it is impossible to accidentally delete data next time?

■ Common Beginner Confusions

Confusion: If I buy DevOps tools, I have DevOps.

Correction: This is Cargo Cult DevOps. Buying Jenkins, Docker, or Kubernetes without changing the culture will fail. The mindset shift must come first.

✓ Quick Revision Summary

Culture > Tools — always.

C.A.L.M.S.: Culture, Automation, Lean, Measurement, Sharing.

Embrace failure as learning (Blameless Post-Mortems).

Automate everything possible to reduce toil.

Measure performance and use data to continuously improve.

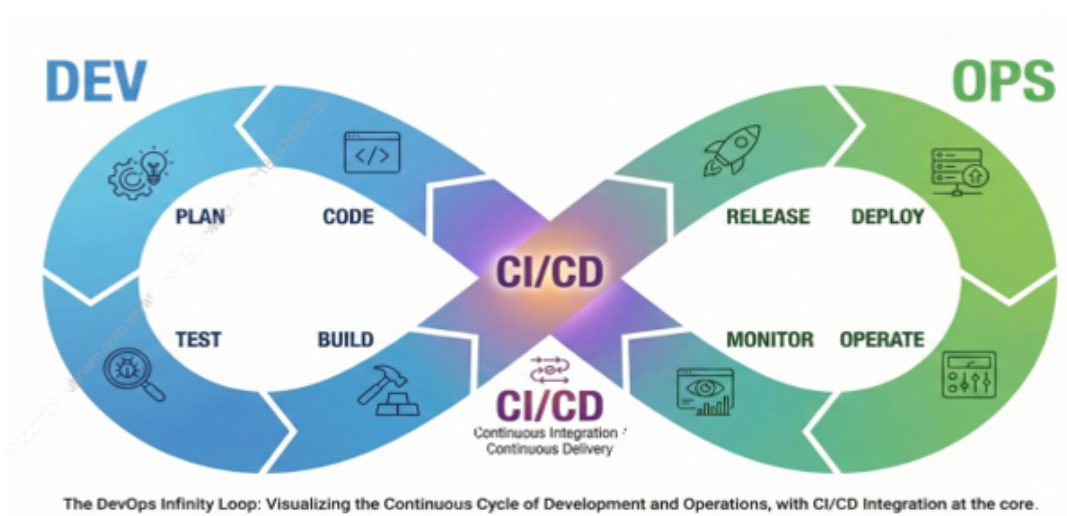
CHAPTER 05

The DevOps Lifecycle

05

1. Simple Explanation

Software never ends. It is a continuous loop of improving and updating. The DevOps Lifecycle maps the journey of a piece of code from an idea to a running application — and back again as feedback drives the next iteration.



The DevOps Infinity Loop — Plan, Code, Build, Test, Release, Deploy, Operate, Monitor

2. The 8 Stages

Stage	Purpose	Example Tools
Plan	Decide what to build.	Jira, Confluence, GitHub Issues
Code	Write the software.	VS Code, Git, GitHub / GitLab
Build	Package code into an executable.	Maven, Gradle, npm
Test	Check for bugs automatically.	JUnit, Selenium, SonarQube
Release	Approve the changes.	Semantic versioning, change management
Deploy	Put it on the live servers.	Jenkins, GitHub Actions, ArgoCD
Operate	Keep it running.	Kubernetes, Docker, Ansible
Monitor	Watch for errors and feedback.	Prometheus, Grafana, Datadog

3. Real-World Example

A game developer Plans a new level, Codes it, Builds the game file, Tests it for glitches, Releases it to the app store, ensures servers handle the players (Operate), and checks if players are crashing at a particular

point (Monitor). That monitoring data feeds back into the next Plan.

✓ Quick Revision Summary

It is a continuous loop, not a straight line.

Left side (Dev heavy): Plan, Code, Build, Test.

Right side (Ops heavy): Release, Deploy, Operate, Monitor.

Monitoring feeds back into Planning — closing the loop.

CI/CD sits at the heart, automating the handoffs.

CHAPTER 06

CI/CD Concept (High Level)

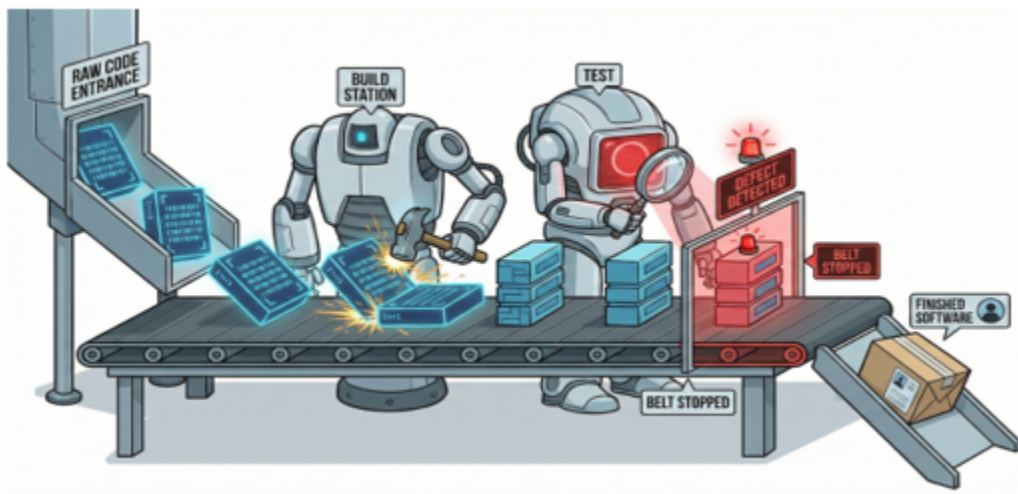
06

1. Simple Explanation

CI — Continuous Integration: Developers save their work to a shared repository often — daily or multiple times a day. An automatic robot checks if new work breaks any existing work.

CD — Continuous Delivery: If the robot says the work is good, it automatically packages it and prepares it for release — one button click away from being live.

CD — Continuous Deployment: Goes one step further — no human button needed. If all tests pass, the code goes live automatically.



CI/CD as a factory assembly line: Build Station → Test Station → Finished Software

2. Why This Matters

- It stops integration hell — where merging months of separate work takes weeks of fixing conflicts.
- It allows companies to release updates multiple times a day instead of once a year.
- It ensures every change is tested — making releases dramatically safer.

3. Key Concepts

Pipeline: The set of automated steps code goes through: Commit → Build → Test → Deploy.

Build Automation: Compiling and packaging code automatically — no manual steps.

Automated Testing: Running unit, integration, and end-to-end tests without human input.

Fail Fast: Detect problems as early as possible in the pipeline — cheap fixes versus expensive production incidents.

4. Real-World Example

When Facebook updates its app colour, a developer commits the one-line change to Git. GitHub Actions sees the change, runs a thousand automated tests in parallel, passes them all in under 5 minutes, and

updates the app on your phone automatically — all without a single human touching a server.

✓ Quick Revision Summary

CI: Merge code often, test automatically.

CD (Delivery): Release reliably at any time — human approves.

CD (Deployment): Code goes live automatically if all tests pass.

A Pipeline automates the entire path from commit to production.

Fail Fast: Find bugs early when they are cheap to fix.

CHAPTER 07

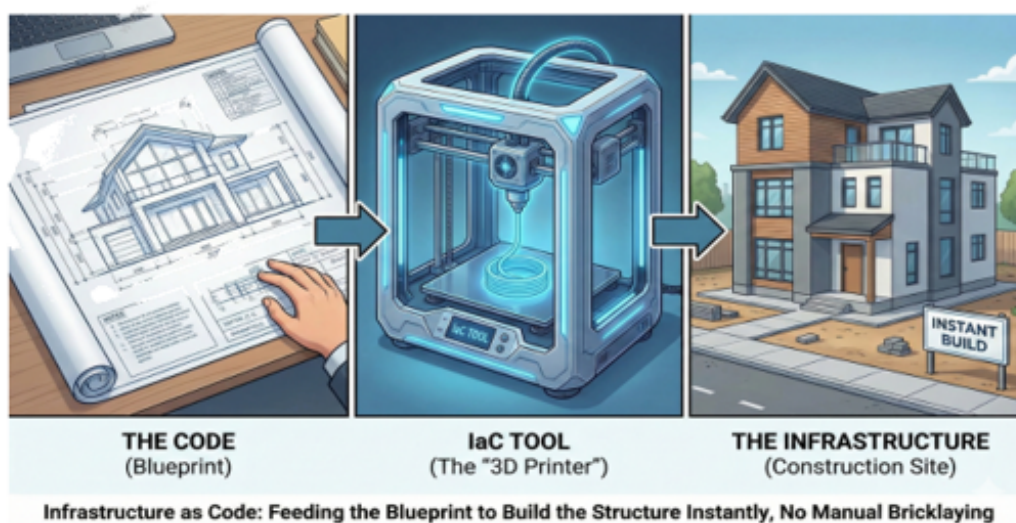
Automation & Infrastructure as Code (IaC)

07

1. Simple Explanation

Automation: Using tools and scripts to do repetitive tasks so humans do not have to — and so they are done the same way every single time.

Infrastructure as Code (IaC): Instead of physically clicking buttons to set up a server, you write a text file (script) that describes the server. The computer reads the file and creates the server instantly and repeatably.



IaC: The Blueprint (code) goes into an IaC tool and produces the Infrastructure

2. Why This Matters

- Manual setup leads to errors — humans make mistakes, especially on the 50th server.
- Automation makes things consistent — same script, same result, every time.
- Dramatically faster — provisioning 50 servers takes seconds versus days of manual work.
- You can version control your infrastructure just like your application code.

3. Key Tools

Tool	What it does
Terraform	Defines what cloud infrastructure should exist. Works with AWS, Azure, GCP.
Ansible	Configures software on servers that already exist. Agentless.
Chef / Puppet	Older config management tools. Agent-based on target servers.
AWS CloudFormation	Terraform equivalent but only for AWS resources.

4. Real-World Example

A company needs 50 servers for a Black Friday sale. Instead of hiring 10 people to click through the AWS console for a week, they run one Terraform script. 50 servers appear in 4 minutes, identically configured, ready for traffic. After Black Friday, one more command destroys them all — stopping the billing immediately.

✓ Quick Revision Summary

Treat infrastructure (servers, networks) like software code.

Removes human error — same script, same result, every time.

Enables quick replication of environments (Dev, Staging, Production).

Key tools: Terraform (provision), Ansible (configure).

Version control your infrastructure — git blame works on servers too.

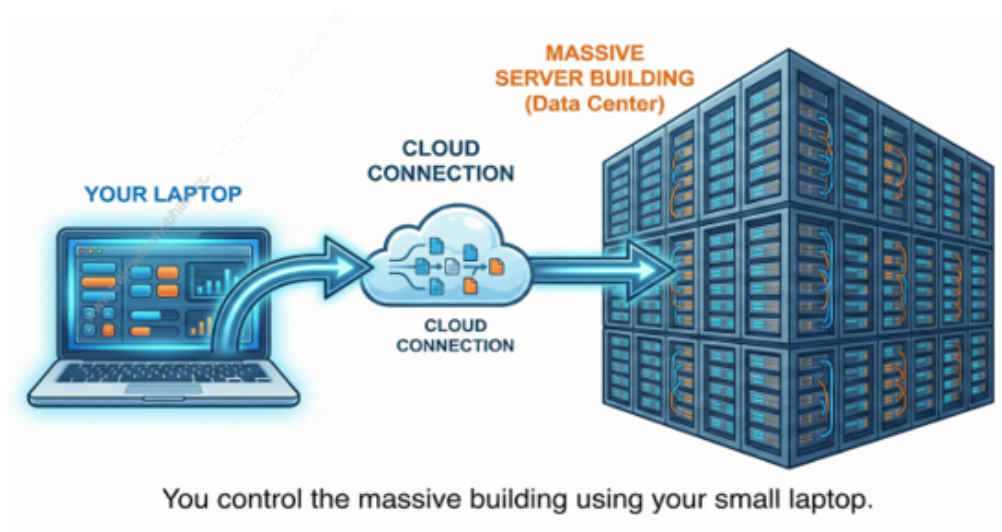
CHAPTER 08

What is Cloud Computing

08

1. Simple Explanation

Cloud computing is renting someone else's computer over the internet. Instead of buying a server and putting it in your office closet, you pay Amazon, Microsoft, or Google to use their massive data centers. You control everything via a web browser or command line — from anywhere in the world.



Cloud Computing: on-demand access to computing resources over the internet

2. Why This Matters in DevOps

- You do not have to buy expensive hardware upfront (CapEx vs OpEx model).
- You can access and provision resources from anywhere in the world in minutes.
- Cloud providers handle the physical maintenance — you focus on building.
- It makes IaC and CI/CD possible at scale — spin up a test environment in seconds.

3. Key Concepts

On-Demand: Get a server whenever you want. No waiting for procurement approval.

Pay-as-you-go: Pay only for what you use — like an electricity bill, not a mortgage.

Elasticity: Automatically grow and shrink resources based on demand.

Managed Services: Cloud providers offer databases, AI, monitoring, and security as managed products.

4. Real-World Example

Netflix does not own data centers. It rents massive computing power from AWS (Amazon Web Services) to stream movies to 260 million users simultaneously. Netflix can scale up capacity during peak hours and scale down at 3am — paying only for what they use.

■ Common Beginner Confusions

Confusion: Is the cloud actually in the sky?

Correction: No. It is physical hardware in massive, secure warehouses called data centers.

Confusion: Cloud means my data is safe.

Correction: Cloud providers protect the infrastructure. You are responsible for securing your data, configurations, and access controls (Shared Responsibility Model).

✓ Quick Revision Summary

Computing power delivered over the internet — rented, not owned.

No hardware maintenance for you to worry about.

Pay only for what you use — scale instantly.

Makes DevOps practices possible at any scale.

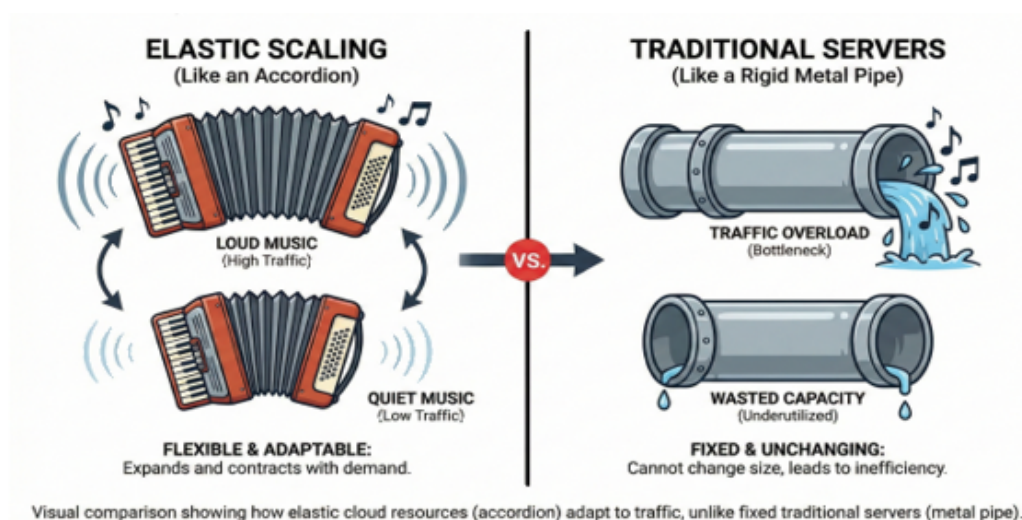
CHAPTER 09

Why Cloud is Needed

09

1. Simple Explanation

Traditional servers are expensive and hard to capacity-plan. If you buy too few servers, your site crashes under traffic. If you buy too many, you waste money on idle hardware. Cloud solves this by letting you change your server count instantly — expanding and contracting with demand like an accordion.



Cloud elasticity: scale up during peak demand, scale down during quiet periods

2. Key Benefits

Scalability: Ability to grow or shrink resources based on traffic demand — automatically.

Speed: Developers can get a new server in 2 minutes, not 2 weeks waiting for procurement.

Reliability: Cloud providers have backup power, redundant networking, and 99.99% uptime SLAs.

Elasticity: Automatic scaling up during peak and down during quiet periods.

Cost Efficiency: No wasted resources sitting idle. No upfront capital expenditure on hardware.

Global Reach: Deploy code to data centers in Japan, US, and Europe in one click.

3. Real-World Example

Zoom experienced massive, unexpected growth during the COVID-19 pandemic — going from 10 million to 300 million daily meeting participants in weeks. If they had relied on physical servers, they would have failed. Using AWS cloud infrastructure, they scaled up capacity within hours to support millions of new users without a service outage.

■ Common Beginner Confusions

Confusion: Cloud is always cheaper than on-premise.

Correction: Not always. If you leave servers running 24/7 when you do not need them, cloud bills become enormous. Cloud requires active cost management.

✓ Quick Revision Summary

Handles traffic spikes (Scalability and Elasticity).

Lowers upfront cost — OpEx instead of CapEx.

Increases innovation speed — get resources in minutes.

Global availability — deploy anywhere instantly.

Requires cost management — idle resources still bill.

CHAPTER 10

Cloud Service Models — IaaS, PaaS, SaaS

10

1. The Pizza Analogy

IaaS — Infrastructure as a Service: Take and Bake. You rent the raw kitchen and oven. You buy the ingredients and cook it yourself. You manage the OS, runtime, and application.

PaaS — Platform as a Service: Pizza Delivery. The platform brings a pizza to your door. You add your own toppings (code). The platform manages the OS, runtime, and middleware.

SaaS — Software as a Service: Dining Out. You sit down, order from the menu, and eat. Everything is managed for you.



The pizza analogy: On-Premise (made from scratch), IaaS, PaaS, SaaS (dining out)

2. Real Services

Model	Examples	You manage
IaaS	AWS EC2, Azure VMs, Google Compute Engine	OS, Runtime, Apps, Data
PaaS	Heroku, Google App Engine, Elastic Beanstalk	Applications and Data only
SaaS	Gmail, Salesforce, Dropbox, Slack	Your settings and data only

3. Real-World Examples

IaaS: A startup rents AWS EC2 virtual machines to host a custom database and web server. Full control, full responsibility.

PaaS: A developer uploads Python code to Heroku to run a blog without ever setting up a server.

SaaS: An HR team uses Salesforce to track employees. They never touch a server — they just log in and use the app.

■ Common Beginner Confusions

Confusion: Is AWS IaaS or PaaS?

Correction: AWS offers both. EC2 is IaaS (you manage the OS). Elastic Beanstalk is PaaS.

Lambda is FaaS (Function as a Service). It depends on the specific service you select.

✓ Quick Revision Summary

IaaS: Infrastructure — Host it yourself on rented hardware.

PaaS: Platform — Build on it without managing infrastructure.

SaaS: Software — Just consume it.

Key trade-off: More control (IaaS) vs more convenience (SaaS).

CHAPTER 11

Cloud Deployment Models

11

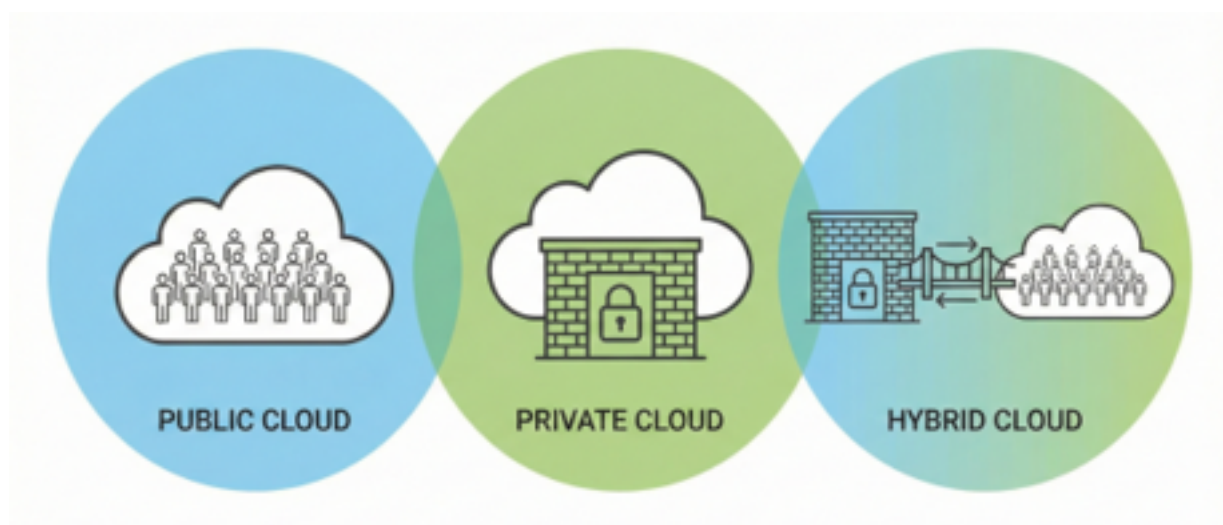
1. Simple Explanation

Public Cloud: Like a public bus. Shared by many companies, cheap, efficient. AWS, Azure, GCP.

Private Cloud: Like a personal car. Only for your organisation, secure, expensive to maintain.

Hybrid Cloud: Like a taxi. You have your private space, but use public infrastructure when needed.

Multi-Cloud: Using multiple cloud providers simultaneously — for redundancy or best-of-breed services.



Visual comparison of Public, Private, and Hybrid Cloud models

2. Key Concepts

Public: AWS, Azure, Google Cloud (GCP). Resources shared across many tenants, isolated by virtualisation.

Private: Servers dedicated to one company — often on-premise or in a colocation facility.

Hybrid: Connecting Public and Private clouds together via a secure network link (VPN or Direct Connect).

Multi-Cloud: Using AWS for compute, Azure for AI services, and GCP for BigQuery analytics simultaneously.

3. Real-World Example

A major bank keeps customer financial records in a Private Cloud (strict security and compliance regulations) but uses a Public Cloud to run their public-facing website and email marketing (cheaper and faster to scale). This is a classic Hybrid Cloud deployment.

✓ Quick Revision Summary

Public: Shared resources, cost-effective, managed by provider.

Private: Dedicated resources, high security, you manage it.

Hybrid: Best of both worlds — secure data + flexible compute.

Multi-Cloud: Using multiple vendors for redundancy or best-of-breed.

Most large enterprises use Hybrid or Multi-Cloud today.

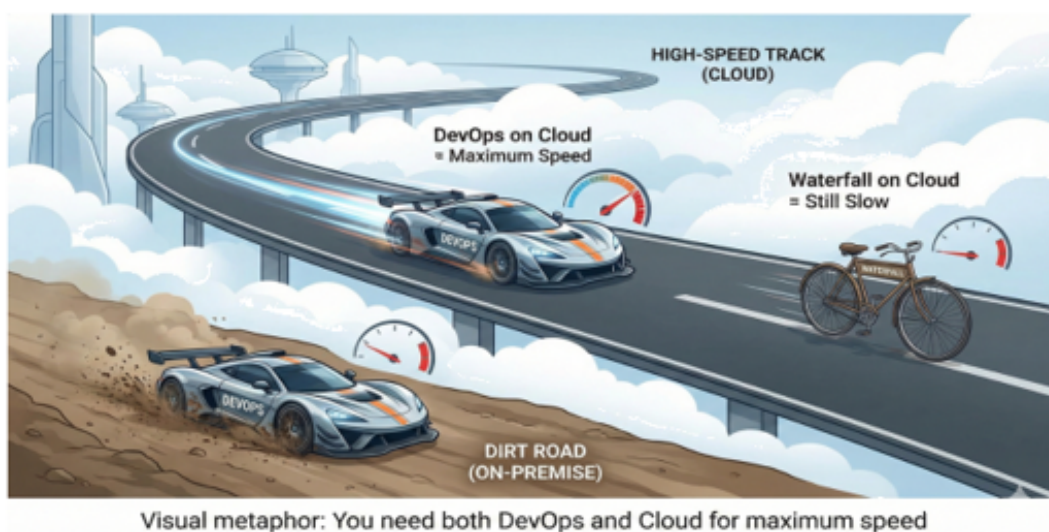
CHAPTER 12

DevOps + Cloud Together

12

1. Simple Explanation

DevOps is the Method — the way you work. **Cloud is the Toolbox** — the place where you work. You can do DevOps without Cloud (but it is hard). You can use Cloud without DevOps (but it is slow). Together, they are a Power Couple for speed, reliability, and innovation.



DevOps + Cloud: Method meets Infrastructure for speed and reliability

2. Why This Matters

- Cloud allows DevOps to automate infrastructure (IaC) at any scale — 1 server or 10,000 servers.
- Cloud provides the unlimited, on-demand resources needed for continuous testing and deployment.
- DevOps practices (CI/CD, monitoring, automation) make Cloud usage efficient and cost-controlled.

3. Key Concepts

Cloud-Native: Building applications specifically designed to run on cloud platforms — using containers, microservices, and managed services.

Programmable Infrastructure: The cloud API allows hardware to be controlled by code — the foundation of IaC.

Serverless: Run code without managing any servers at all (AWS Lambda, Azure Functions).

GitOps: Using Git as the single source of truth for both application code AND infrastructure.

4. Real-World Example

Netflix uses DevOps practices (automated testing, chaos engineering, continuous deployment) on AWS Cloud to update their streaming service globally without any downtime — serving 260 million subscribers. When they deploy a new codec, it rolls out to 100% of users within hours, tested automatically and rolled back instantly if error rates spike.

■ Common Beginner Confusions

Confusion: DevOps and Cloud are the same thing.

Correction: DevOps is HOW you work (methodology, culture, automation).

Cloud is WHERE you work (the computing environment). They are complementary, not synonymous.

✓ Quick Revision Summary

Cloud enables DevOps automation at any scale.

DevOps optimises Cloud usage and controls costs.

Together they enable faster time-to-market.

Cloud-Native + DevOps = the modern engineering standard.

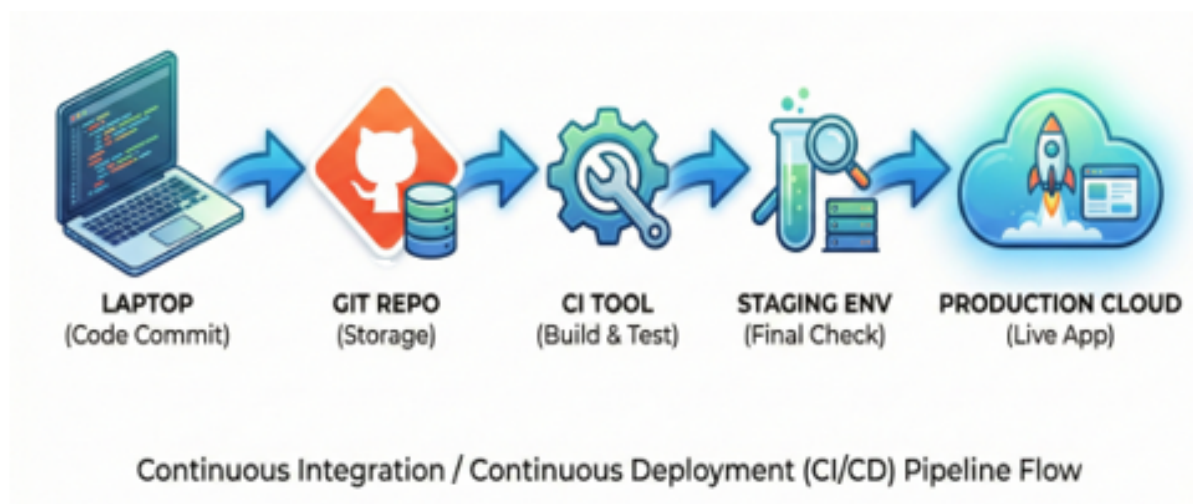
CHAPTER 13

Real-World DevOps & Cloud Workflow

13

1. A Typical Day in the Life of Code

Here is how a piece of code travels from a developer's laptop to a cloud server through an automated pipeline — the exact workflow you will build and manage in a DevOps role:



End-to-end DevOps pipeline: from developer laptop to production cloud

2. The Pipeline Steps

#	Stage	Description
1	Developer Laptop	Engineer writes code locally, tests it manually, and commits to a feature branch.
2	Git Repository	Code is pushed to GitHub/GitLab. A pull request is opened. Teammates review the code.
3	CI Server	Jenkins or GitHub Actions detects the commit, pulls the code, and runs build and all automated tests.
4	Artifact Registry	If tests pass, CI server packages the code into a Docker image and pushes it to a container registry.
5	Staging Environment	The image is deployed to staging on the cloud. Final integration and smoke tests run.
6	Production Cloud	If staging passes, deployment is promoted to production — automatically or with one-click approval.
7	Monitoring	Prometheus and Grafana watch the production system. If error rates spike, an alert fires.

3. Tools in This Workflow

Stage	Tool Examples
Source Control	Git, GitHub, GitLab, Bitbucket
CI Server	Jenkins, GitHub Actions, GitLab CI, CircleCI
Containerisation	Docker, Podman
Artifact Registry	AWS ECR, Docker Hub, GitHub Container Registry
Deployment	Kubernetes, Helm, ArgoCD, AWS ECS
Monitoring	Prometheus, Grafana, Datadog, CloudWatch
Alerting	PagerDuty, OpsGenie, Slack integrations

4. Real-World Example

A developer fixes a typo on the website. They push the change to Git. GitHub Actions sees the change, runs 200 automated tests in parallel in 3 minutes, and updates the website on AWS S3 automatically — in under 5 minutes from the push, without anyone touching a server.

✓ Quick Revision Summary

Code → Repository → CI Pipeline → Testing → Deployment.

Everything is automated where possible — no manual server touching.

Feedback loops return to the developer within minutes, not days.

This is exactly the workflow you will build in a DevOps job.

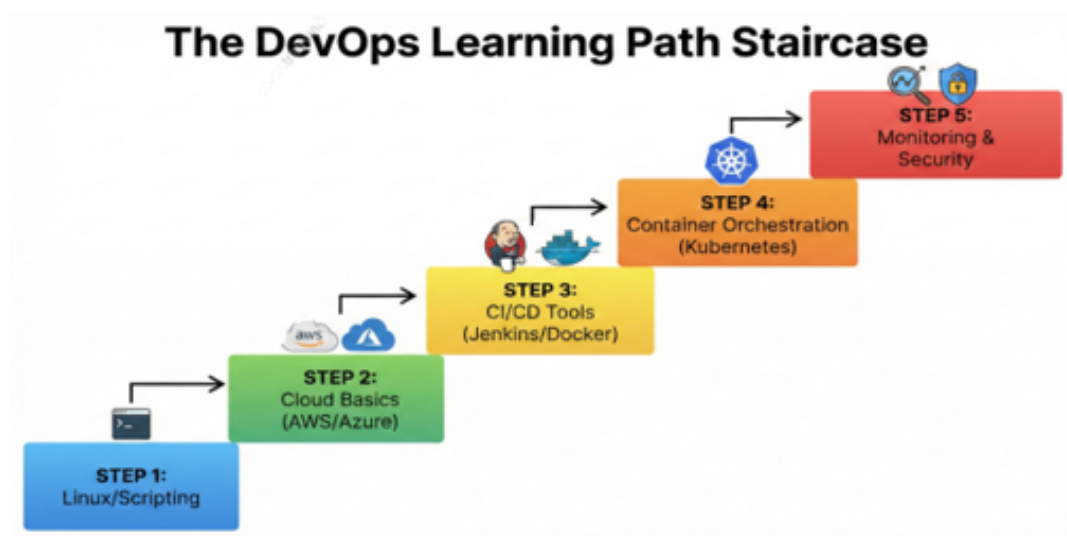
CHAPTER 14

Career Path Overview

14

1. Simple Explanation

DevOps is a journey, not a destination. You usually start by knowing either Development or Operations, then learn the other side. The path is a staircase — each step builds on the last, and you do not need to master everything before taking the next step.



The DevOps career staircase — from Linux basics to Cloud Architect

2. The 5-Step Learning Staircase

Step 1	Linux & Scripting Learn the terminal. Master bash scripting. Understand file permissions, processes, networking. This is the foundation of everything — every server runs Linux.
Step 2	Cloud Basics (AWS / Azure) Get comfortable with one cloud provider. AWS is recommended for job market demand. EC2, S3, IAM, VPC, and basic networking.
Step 3	CI/CD Tools (Jenkins / Docker) Learn Docker containers. Build a Jenkins pipeline. Understand GitHub Actions. This is the core of a DevOps Engineer's daily work.
Step 4	Container Orchestration (Kubernetes) Deploy and manage containerised applications at scale. Kubernetes is the industry standard for production container management.
Step 5	Monitoring & Security Implement observability with Prometheus and Grafana. Understand DevSecOps — security scanning, secrets management, compliance.

3. DevOps Roles in Industry

Role	Focus	Typical Background
Junior DevOps	Linux, scripting, Git, basic CI/CD pipelines	Fresh graduate, system admin, developer
Senior DevOps	Cloud architecture, security (DevSecOps), automation	3–5 years experience
SRE	Reliability, scaling, incident response, error budgets	Strong coding background
Platform Engineer	Building internal developer platforms and tooling	Senior DevOps + product thinking
Cloud Architect	Designing multi-cloud, hybrid solutions for enterprises	10+ years, certifications, breadth

4. Real-World Example

A System Administrator who manages servers starts learning Python and AWS in their spare time. They automate their daily tasks with scripts. They learn Docker and build a Jenkins pipeline for a personal project. Six months later, they are hired as a Junior DevOps Engineer — doubling their salary.

■ Common Beginner Confusions

Confusion: I need to know everything before I apply.

Correction: No. Start with the basics — Linux + one Cloud Provider.

You learn the rest on the job. Employers hire for fundamentals and attitude.

Confusion: DevOps is only for experienced engineers.

Correction: Many people enter DevOps from a fresh start. Start today.

✓ Quick Revision Summary

Start with fundamentals: Linux and Networking.

Learn one Cloud Provider deeply (AWS is recommended).

Master CI/CD concepts — the core of the role.

Container orchestration (Kubernetes) is the next level.

Always keep learning — DevOps evolves constantly.

The best time to start is today.